

Benchmarking LLM-Based Static Analysis for Secure Smart Contract Development: Reliability, Limitations, and Potential Hybrid Solutions

1st Susan Ștefan-Claudiu

Department of Computer Science
Alexandru Ioan Cuza University of Iași
Iași, Romania
claudiu_susan@yahoo.com

2nd Arusoaie Andrei

Department of Computer Science
Alexandru Ioan Cuza University of Iași
Iași, Romania
andrei.arusoaie@uaic.ro

3rd Lucanu Dorel

Department of Computer Science
Alexandru Ioan Cuza University of Iași
Iași, Romania
dorel.lucanu@uaic.ro

Abstract—The irreversible nature of blockchain transactions makes the identification of smart contract vulnerabilities an essential requirement for secure system development. While Large Language Models (LLMs) are increasingly integrated into developer workflows, their reliability as autonomous security auditors remains unproven. This paper investigates the potential of LLMs in identifying vulnerabilities within Solidity smart contracts. We assess whether current generative models are a viable replacement for, or only a complement to, traditional static-analysis tools.

Our findings indicate that LLM efficacy is undermined by both inherent lexical bias and a lack of rigorous validation of external data inputs. This reliance on non-semantic heuristics, such as identifier naming, leads to a high frequency of false positives. Furthermore, prompting techniques reveal a trade-off between precision and recall. These results were derived using our custom automated framework, which achieves 92% accuracy in classifying model outputs.

Index Terms—vulnerability detection, static analysis, large language models, smart contracts, solidity, benchmarking

I. INTRODUCTION

The implementation of smart contracts on blockchain platforms has introduced a “code is law” paradigm, where software vulnerabilities in immutable code lead to financial losses. Traditional static analysis tools such as Slither [9] and Mythril [22] have long served as the first line of defense; however, they are often constrained by rigid rule sets when faced with complex, non-standard code logic. On the other hand, LLMs offer a promising alternative for automated auditing due to their vast training data and ability to generalize to new smart contract implementations.

Integrating LLMs into the blockchain security lifecycle introduces a theoretical paradox. While these models may be able to identify potential exploits, it remains unclear whether they can mimic genuine semantic reasoning or rely solely on probabilistic pattern matching—a phenomenon known as “lexical bias”. Recent findings from Apple [37], show that LLM performance degrades notably when encountering superficial variations in problem descriptions, suggesting an overreliance on lexical cues rather than robust logical derivation.

In the context of developing secure smart contracts, a detector that relies on variable names rather than execution logic is not reliable. This reliance (see, e.g., [39]) makes an LLM a “stochastic auditor”: it can be effective when the code matches its training distribution, but it may fail on novel or sanitized logic that lacks descriptive identifiers. This risk is amplified with the shift towards Agentic AI, where LLMs are expected to function as proactive, autonomous agents for decision-making. To investigate the potential for LLMs to augment existing security issues, this study addresses the following research questions:

RQ1: Lexical Bias To what extent do LLMs rely on non-semantic code heuristics, such as descriptive identifier names and comments, rather than semantic execution logic for identifying smart contract vulnerabilities?

RQ2: Reasoning versus Pattern Matching Does Chain-of-Thought (CoT) prompting enhance the logical reasoning capabilities of LLMs in vulnerability detection?

RQ3: Hybrid Integration Performance How does grounding an LLM with security reports from traditional static analysis tools impact the model’s detection range?

RQ4: Hybrid Integration Bias Does grounding an LLM with security reports from traditional static analysis tools introduce a measurable bias toward the tool’s existing findings?

RQ5: Architectural Efficiency What are the performance and reliability trade-offs when deploying LLMs locally using 4-bit quantization compared to full-precision API-based models for security auditing? Local deployment matters because some auditing clients must keep proprietary contracts on premises and therefore cannot send them to third-party APIs, a common situation in enterprise treasuries, regulated DeFi projects, and pre-disclosure audits.

We conduct an exploratory empirical investigation to extract patterns and anti-patterns that can inform hybrid LLM-assisted smart contract auditing solutions.

Smart contracts are an ideal case study for vulnerability detection using static analysis techniques because they are

specialized programs executed in a distributed environment (the blockchain), where code vulnerabilities can result in significant financial losses. Moreover, once deployed, such programs are public and cannot be modified, making the a priori discovery of vulnerabilities highly desirable. Existing vulnerable smart contract benchmarks often contain suggestive names (for variables, functions, even contracts) that could hint to LLMs about what they should look for. Measuring LLM detection performance may be of limited value, since syntactic clues can inflate the results. This motivates our paired-variant protocol (see Approach below).

Answering our research questions requires addressing several challenges: finding *a relevant benchmark for using LLMs in static analysis* – comparable to the existing ones that were designed for traditional static tools; leveraging LLMs to provide *consistent structured output* – even when explicitly requested to adhere to a strict response format, structural inconsistencies can still be encountered, or receive answers with a mismatching terminology; dealing with *ambiguity in evaluation criteria* – what counts as a correct detection can be unclear; tackling *prompt sensitivity and instability* – LLM behavior varies dramatically, depending on prompt phrasing; acknowledging *scale and cost constraints* – evaluating many LLMs is resource-intensive.

Approach: To address these challenges, we chose an existing collection of smart contract vulnerabilities with a distinctive characteristic: each test item in this collection consists of a pair of smart contracts. Each pair includes a *positive* variant, which contains a vulnerability, and a *negative* variant, which is very similar to the positive variant but does not include that specific vulnerability. This feature allows us to assess the robustness of LLMs for vulnerability detection, as we can verify whether the detector reports the error in the positive variant and does not report an error in the negative variant. In other words, this collection enables us to evaluate whether, *under very similar conditions*, an AI model can precisely identify vulnerabilities. This makes the collection well-suited for studying the limitations of LLM-based detection.

Our evaluation of the LLMs consists of prompting the models with different techniques, both with and without information from external tools, for each positive/negative variant. We built a reusable infrastructure that allows the experiments to be run consistently and facilitates the integration of additional models. Since we already have positive/negative variants for each smart contract in the collection, we assign labels to each output to indicate whether it is a *true positive* (TP), *true negative* (TN), *false positive* (FP), or *false negative* (FN). We performed this labeling manually for several tests, but we also developed a method to *automatically* assign these labels with *high* accuracy.

Contributions: This study makes the following contributions:

- 1) An exploratory empirical characterization of how *lexical bias*, *prompting*, and *external integrations* affect LLM-based vulnerability detection in smart contracts.

- 2) A large-scale benchmarking evaluation of 15 LLMs, establishing a realistic baseline for AI-enhanced smart contract auditing.
- 3) A methodological contribution in the form of a paired-variant lexical-bias protocol applied at scale, which compares positive and negative variants under matched conditions to isolate superficial lexical cues.
- 4) An explicit set of answers to **RQ1–RQ5**, obtained by comparing matched positive and negative contract variants under controlled prompting and external-tool conditions, and by analyzing the resulting model outputs with our classification pipeline.
- 5) An engineering contribution in the form of a reusable reproduction infrastructure and an automatic classifier for unstructured LLM outputs, which achieves 92% accuracy and enables reproducible large-scale experiments on LLM-based code analysis.

Paper Organisation: In Section II, we provide a brief literature review of closely related work. We also present the collection of smart contracts used in our study, along with the main definitions and metrics that we adopt from existing research. In Section III, we describe our methodology and the infrastructure that we created. In Section IV, we present the evaluation results that we obtained. In Section V, we highlight our key findings and discuss our future plans.

II. PRELIMINARIES

A. Smart Contracts and Blockchain

Smart contracts [5] are programs executed within a decentralized environment known as the blockchain. These contracts often handle valuable assets, including cryptocurrencies and other types of digital tokens. One of the major advantages of smart contracts is their ability to encode agreements that are automatically executed. The consensus mechanisms implemented by blockchains ensure some degree of immutability, meaning that once deployed, smart contracts cannot be modified or altered. This also ensures transparency, as the code is publicly available for anyone, making all actions traceable.

However, this immutability comes with a significant downside: it is extremely risky to deploy buggy or vulnerable contracts. A vulnerability in a smart contract could lead to serious financial and operational consequences. Additionally, since the code is publicly available, it is more exposed than traditional software, making it an attractive target for malicious actors. Some examples of such vulnerabilities and their consequences can be found in [3], [4], [16], [20], [24].

B. Prompting

Zero-Shot prompting is an interaction paradigm where an LLM is tasked with performing a specific function—such as identifying a smart contract vulnerability—without receiving prior examples or task-specific demonstrations [46]. This technique serves to evaluate the model’s inherent ability to generalize its pre-trained knowledge and interpret code logic across diverse software domains [7].

Chain-of-Thought (CoT) prompting is a strategy designed to elicit logical reasoning by instructing the model to generate intermediate analytical steps before arriving at a final output [47]. By mimicking a human-like, step-by-step deductive process, CoT aims to improve model performance on complex tasks, such as tracing a reentrancy path through inter-contract calls. However, recent findings suggest that while reasoning-intensive approaches effectively filter out superficial noise, they often introduce a trade-off by narrowing the overall discovery range and increasing computational latency [37].

The interaction with Large Language Models is architecturally segmented into system and user prompts to separate behavioral meta-rules from specific task data. The system prompt serves as a foundational configuration layer, defining the model’s persona, expertise, and persistent operational constraints—such as formatting requirements and tone—following established patterns [29], [33]. In contrast, the user prompt acts as the dynamic input layer, providing the specific, variable content or query intended to drive a particular response for a single request. By isolating operational directives from task-specific inputs, this dual-layer architecture ensures consistent behavioral alignment and performance across complex applications.

C. LLM Inference Internals

The Key-Value (KV) cache is a memory optimization mechanism used during the inference of Transformer-based models to store the intermediate mathematical representations of the self-attention layers [40]. In autoregressive generation, each new token is generated based on the context of all preceding tokens; the KV cache stores the “Keys” and “Values” of these past tokens to avoid redundant computations, enabling the model to process only the most recent token at each step [45]. However, as the cache size increases linearly with sequence length, it can place substantial pressure on the system’s video memory (VRAM), potentially leading to performance degradation or hardware failures during long-context analyses [35].

D. LLM Inference Hyperparameters

Temperature is a hyperparameter utilized during Large Language Model inference to adjust the randomness or “creativity” of the generated output. It functions by scaling the probability distribution of the next-token prediction; a value of zero makes the model deterministic by ensuring it consistently selects the token with the highest probability, whereas higher values encourage the selection of less likely tokens to produce more diverse responses [1]. This parameter is critical for balancing the model’s output between strict adherence to learned patterns and exploratory generation.

Structured output refers to the enforcement of a rigid, predefined data format—typically JSON—on the response generated by a Large Language Model to ensure its compatibility with downstream software systems. This mechanism is designed to eliminate non-essential conversational text and ensure that all findings follow a strict schema, which is

essential for the reliability of automated parsing and classification pipelines. LLM providers facilitate this through two methods: request parameterization (schema enforced natively at inference) and instructional enforcement (system prompt constrains the model’s linguistic behavior) [29]. By converting probabilistic text into deterministic data structures, structured output allows generative models to function as reliable components within complex, programmatic workflows.

E. Embeddings and Semantic Similarity

Embeddings are dense, high-dimensional vector representations of textual data where words, phrases, or code snippets are mapped into a continuous mathematical space [17], [19]. This transformation allows the model to capture deep semantic relationships, as tokens with similar meanings or functional roles are positioned in close proximity to one another [40].

Semantic similarity is a measurement of the conceptual correspondence between data objects—such as words, phrases, or code snippets—within a high-dimensional mathematical space [6]. This approach moves beyond simple lexical or character-level matching by evaluating the proximity of data points based on their underlying meaning as represented by embedding vectors. Unlike discrete representations, these embeddings enable vector-space operations to identify patterns and quantify conceptual similarity (e.g., cosine similarity or Euclidean distance).

F. Model Compression

Model quantization is a compression technique that reduces the numerical precision of a model’s weights and activations—typically from 16-bit or 32-bit floating-point numbers to lower-bit formats like 4-bit or 8-bit [41], [42]. By using fewer bits to represent each parameter, this process significantly decreases the overall memory footprint and VRAM requirements needed for inference [43]. While effective for optimizing deployment on limited hardware, the reduction in mathematical resolution can introduce quantization-induced logic loss, potentially impairing the model’s capacity for complex logical reasoning [44].

G. Related Work

The consequences of vulnerable smart contracts code have motivated the research community and practitioners to develop detection tools and collections of vulnerable smart contracts. Currently, there are many such collections [2], [10]–[12], [14], [18], [26], [27], [31], [34], most of which are written in Solidity [25], the most popular smart contract language. These collections are organized in different ways, depending on their intended purpose (e.g., [2], [11], [18], [26], [27]). For example, [18] and [27] are community-developed collections primarily aimed at documentation, while also providing a taxonomy of vulnerabilities. In [26], the tests are curated and crafted to be reproducible using Foundry. The collection in [11] is organized according to the taxonomy from [18].

TABLE I
DEFECT CATEGORIES

Category Index	Category Name
1	Malicious Environment, Transactions or Input
2	Blockchain/Environment Dependency
3	Exception & Error Handling Disorders
4	Denial of Service
5	Resource Consumption & Gas Issues
6	Authentication & Access Control Vulnerabilities
7	Arithmetic Bugs
8	Bad Coding and Language Specifics
9	Environment Configuration Issues
10	Eliminated/Deprecated Vulnerabilities

Selected Benchmark: The collection that we propose in [2] features a characteristic that enables a more robust evaluation of vulnerability detectors. This collection follows a comprehensive taxonomy from [21] and is inspired by the Toyota ITC Benchmark [23]. The categories of vulnerabilities in this taxonomy are shown in Table I. The distinctive feature of this collection is its use of pairs of tests that are mostly identical, with one test containing a vulnerability (a.k.a., the *positive* variant) and its counterpart being nearly identical but free from the vulnerability (a.k.a., the *negative* variant). In [2], the authors exploit this feature to compare existing vulnerability detectors in a robust way: a detector should only signal the vulnerability on the positive variant. A relevant question is whether AI models, which rely on probabilistic methods, can distinguish between the two variants. Moreover, a comparison of several detectors based on Detection Rate, False Positive Rate, Accuracy, Unique Findings, Precision, Recall, and F-scores is provided. In this work, we use some of these metrics to compare AI models’ detections against each other.

1) *Similar Approaches:* We highlight two approaches closely related to ours and the differences between our work and theirs. In [8], the authors use the GPT [1] and Llama 2 [28], models available in 2023, to detect vulnerabilities in the Smartbugs curated dataset [11]. They considered only 7 types of vulnerabilities in order to evaluate how effective they are at detecting vulnerabilities. Their main conclusion is that while ChatGPT demonstrated strengths in pinpointing certain vulnerabilities, it also encountered precision-related challenges. A major difference from their work is that we consider more models and a significantly different benchmark that enables us to identify robust detections.

A key difference in prompt design is that the approach in [8] also includes the expected vulnerability categories. The prompt we use includes instructions that guide the model towards returning the detections in a specific format, which helps us parse the detections and store them in a database. The authors of [8] experimented with guiding the models towards a specific format, which reduced the quality of detections.

Since 2023, there have been great improvements in the capabilities of LLMs to return structured outputs, driven in large part by the wide adoption of Agentic AI tools. Although not true for all the models that we tested, some managed to adhere to the desired response structure for all queries.

The results in [8] show that LLMs achieve a satisfactory recall rate and a large false positive rate. This outcome is similar to the one in our 1st set of experiments, but in the rest of our experiments, the recall rate dropped significantly.

Besides measuring the performance of the models, the authors also tested the consistency of responses and identified some causes for the incorrect detections that the LLMs produced.

A more recent study including more models is presented in [30]. The study included five up-to-date models from different providers: GPT-4o-mini, Gemini 1.5 Pro [13], Claude 3.5 Sonnet, Yi-large [32], and Qwen-plus. They used 3 datasets: **TOP200** consists of open-source, well-audited contract projects primarily used for evaluating FPR.

Web3Bugs contains exploitable bugs extracted from audit contests, primarily written in Solidity 0.8, and used to evaluate detection performance across different types of vulnerabilities.

Messi-Q Smart Contract Dataset contains data labeled with specific vulnerabilities, written in Solidity v0.4, and used for comparative experiments with older versions.

One of their most significant findings was the high false positive rate that all models presented, especially Claude 3.5 Sonnet. This finding corresponds with the results obtained in our experiments as well. The authors were able to reduce the number of false detections by designing a prompt that targets one vulnerability at a time. They concluded that having the LLM focus on one task (one vulnerability type) provides a significant decrease in false alarms.

Besides the scale of the experiments, our approach distinguishes itself from similar efforts in the following aspects:

- 1) Our methodology highlights the significant performance volatility that LLMs have depending on code naming conventions, comments, prompting, and external integrations. Thus, quantifying their *lexical bias*, *prompt sensitivity*, and *external data source bias*, concerning aspects regarding the usage of LLMs alone as both standalone detection tools, as well as part of a hybrid system.
- 2) Our automatic detection classification technique enabled us to perform multiple large-scale experiments with the possibility of reuse for similar future community efforts.

III. METHODOLOGY

A. Strategy

Our research methodology is structured into four stages: *the preparation of suitable code examples, designing the prompts, the analysis of the files using our various AI models, and the classification of detections in various defect categories*. The first two stages were performed manually. The large number of models and vulnerability types led us to implement a reusable framework for automating the last two stages.

The methodology flowchart is provided in Appendix A (Figure 3). The study focuses on Ethereum smart contracts, so we chose Solidity, the language that dominates the deployed smart-contract market and is the only language with paired positive-negative data sets. This choice also reflects the

blockchain-specific and language-specific nature of our study and of the benchmark, which are tailored to the execution model, semantics, and tooling ecosystem of smart contracts. In addition, because of its popularity, Solidity is likely to be more frequently represented in LLM training data sets than other smart-contract languages such as Vyper. The methodology transfers in principle to other smart-contract languages. The closest extension is Vyper since it is EVM-compatible and has similar ASTs. Rust-based chains would require a new paired benchmark, and Move would invalidate several defect categories under its resource semantics. Producing such benchmarks is left to future work.

Stage 1, preprocessing the collection of defect examples: We have chosen the test suite presented in [2] due to its wide variety of defect categories and its positive/negative pairs for each vulnerability.

This comprehensive range of defect categories allows us to test the capabilities of LLMs in various scenarios. The negative variants also enable us to test the tendency of models to return false alarms.

In addition, the tests in the selected suite are annotated. The defect contained in each code file (for positive variants) or absent from each file (for negative variants) is clearly indicated in the file name or the name of the vulnerable function. This feature facilitates metric computation. However, it may also bias AI models toward a specific defect type, since annotated defect category names can act as cues. As a result, such detections may not be fair, because these annotations are not present in practice. To avoid this, we performed some additional preprocessing of the dataset, such as renaming the contracts, methods, parameters, or variables. This allows us to measure the lexical bias.

The original suite contains **208** code examples distributed across **10** generic defect categories and **52** specific vulnerability types. To accommodate our automated classification pipeline, we excluded all examples from **3** specific categories (see details in Section III-D). This filtering resulted in the final dataset for our study, comprising **176** code examples across **49** specific defect categories.

Stage 2, designing the prompts: We designed two prompts. The first requests an analysis report from the model without any additional instructions besides asking it to assume a “senior Smart Contract Security Auditor specializing in Solidity” role. For the second, we present the LLM with a structured approach with additional instructions that ask for additional internal validation of the findings. In prompt engineering (see, e.g., [7], [29]), these two prompts correspond to *Zero-Shot* and *CoT* prompting. We focus on these two canonical prompting families, and we exclude few-shot prompting because per-vulnerability exemplars would leak defect-type cues. For each of these initial prompts, we generated a variant in which we provided the model with the security report generated by Slither. For the CoT variant, we added an additional step in which we instruct the model to attempt to disprove the detections provided by Slither in an effort to mitigate potential biases. The resulting prompts are used to perform several

experiments detailed in Section IV. The complete prompts, including the structured output prompt used as a system prompt for models without dedicated schema-enforcement request parameters, are reproduced in Appendix B.

Stage 3, analyse defect examples: We used official client libraries to communicate with both remote and local model endpoints and prompted the models to return structured detections: defect type, general defect definition, and how the defect manifests in the contract being analyzed. The resulting analysis report is then stored in a database. To handle transient API errors, such as server overload or rate limiting, failed analysis runs were stored and subsequently retried. This ensures complete data collection and prevents temporary issues from being incorrectly marked as undetected vulnerabilities.

Stage 4, classify the results of the analysis: This stage begins by automatically assigning specific defect types to the detections generated during the processing stage. Due to the large number of detections that we obtained, manually classifying each detection is not feasible. We describe the process of designing and implementing our automatic classification step in greater detail in Section III-D. Following the assignment of defect types, we assign labels to each output to indicate whether it is a *true positive* (TP), *true negative* (TN), *false positive* (FP), or *false negative* (FN). Then, relevant performance metrics are computed based on the detections and their assigned labels. We also measure the number of instances where an LLM failed to produce structured output and the number of detections that match the ones provided by Slither.

B. Interacting with LLMs

To ensure that LLM outputs were easy to parse, we asked for a strict JSON-structured format. Depending on the provider, we did this using one of the methods below:

- 1) Request Parameterization: For models and providers supporting native schema enforcement (e.g., Grok, Qwen), we integrated the desired response structure into the request parameters to ensure 100% format compliance.
- 2) System Prompt Enforcement: For models lacking native parameter-based JSON support (e.g., Mistral, DeepSeek), we developed a specialized prompt to enforce the schema and suppress non-essential conversational text.

Runtime and formatting benchmarks were set locally using the Ollama framework. The main experimental pipeline used official developer APIs to mitigate hardware-induced performance bottlenecks and provide access to the full operational capabilities of the models. An exception was made for GPT-Oss 120b, which was accessed via OpenRouter due to its unavailability through the main OpenAI API. We set a timeout limit of **5 minutes** for both local and API-based model queries.

C. Automated Benchmarking Framework

To operationalize the methodology described above, we implemented a reusable benchmarking framework that minimizes manual intervention during experiment execution and result collection. The framework was designed around four requirements.

Automation. It executes complete experiment batches end to end, including prompt expansion, model invocation, structured-output validation, persistence of intermediate results, and retry handling for transient failures.

Configurability. It supports parameterized prompt templates in which system prompts and user prompts can be combined in the same experiment, replacement tokens can be defined for contract code and auxiliary inputs, and individual models can be included or excluded without modifying the core execution flow.

Model compatibility. It abstracts remote APIs and local inference backends through a shared execution layer. When a provider requires a dedicated SDK, the framework adapts that SDK to the common interface so that model integration remains uniform.

Scalability. Our initial sequential implementation required approximately **18 hours** to complete a full experiment. By batching API-based models by provider and executing independent jobs concurrently, we reduced the runtime to approximately **10 hours**. On the available hardware, however, parallel execution of multiple local models was not feasible because of memory pressure and inference contention, so local runs remained serialized.

D. Automatic defect detection classification

The approach we propose is grounded in the principles of *Semantic Similarity* [6], which facilitates the measurement of correspondence between data objects within a high-dimensional conceptual space. This allows for a robust classification of model outputs that goes beyond lexical-based comparison. We assigned a definition to each defect. To avoid noise, the definitions were concise, consisting of a few phrases. For example, the category Frozen Ether is described as:

A permanent loss scenario where ether is locked in a contract due to missing withdrawal mechanisms, unreachable conditions (e.g., broken access control), or logic errors preventing legitimate users from retrieving funds.

We measured the accuracy of our classifier by manually classifying the results from 6 models (approx. 4000 detections) and using them as a reference. During the manual classification process, we inspected every detection and checked if the defect category and the description of the vulnerability in the context of the contract matched a category present in the taxonomy that our benchmark is based on.

After experimenting with multiple embedding models such as *nomic-embed-text*, *bge-large3*, *granite-embedding*, we obtained the best accuracy using *gwen3-embedding:8b*.

Our automatic similarity-based classifier works as follows.

Initially, we compute the similarity between the defect type name from our list to the defect type name from the detection. If the similarity threshold required for an assignment is not met, then we also perform the same similarity computation for both the defect and its general definition.

We noticed that, in instances where certain defect categories were misclassified, the assigned category closely matched the expected category.

Usually, they were different instances of the same general defect. For example, detections for Gas Costly Loops were often wrongly assigned as Gas Costly Pattern. To increase the effectiveness of our process, we decided to allow it to accept close matches for similar defects. The resulting close-match pairs are listed in Appendix D, Table VII.

Since we know the defect corresponding to each positive and negative variation, we are assigning that defect to the detection if the classified defect is an exact or a very close match. Otherwise, they remain unassigned to not include out-of-scope detections. By relaxing the assignment constraints in this way, we were able to increase the overall accuracy to over **80%** when compared to the results of manual classification.

However, detections from the defect categories Unexpected Throw and Revert (3B), and Assert, Require or Revert Violation (3D) were still not being properly classified. Due to their relatively large number of samples and a detection classification accuracy of less than **40%**, we decided to exclude samples from these categories from our study. Even though the sample size for Callstack Depth Limit (10C) was small compared to others, we also excluded it due to its low classification accuracy score. This decision increased the accuracy of our automatic detection classification process to **88%**. The test suite is organized in categories and subcategories. The defect classes mentioned above are subcategories. Thus, we preserved **49** out of the **52** initial subcategories, still having examples from all **10** larger categories; the retained subcategories are listed in Appendix C.

We observed that the inherent granularity of defect taxonomies significantly influenced classification accuracy. Specifically, certain categories, such as **8L** (Inadequate Logging or Documentation), function as broad umbrellas covering diverse issues like missing event emissions or unindexed addresses. In several instances, the models correctly identified a specific defect but failed to map it to the broader parent category, returning the granular issue instead of the generic label. To address this semantic granularity gap, we expanded the classifier’s mapping logic by introducing additional synonymous definitions and sub-category identifiers. This refinement better aligned the model’s specific detections with the study’s general taxonomy, ultimately increasing the classifier’s accuracy from **88%** to **92%**.

IV. EVALUATION PROCESS

A. Overview

Given the description of *Stage 2* in our methodology, we evaluate how each combination performs on our dataset: Zero-Shot with and without Slither input, and CoT with and without Slither input. As described in *Stage 1*, we preprocessed the dataset in [2] to eliminate the most basic forms of lexical bias. To differentiate between the original, unprocessed dataset from [2] and the processed one used here, we refer to them

as Dataset 1 and Dataset 2, respectively. The experiments that we perform and their objectives are shown in Table II.

TABLE II
EXPERIMENTAL DESIGN AND OBJECTIVES

Exp.	Dataset	Prompting	Slither	Primary Objective
1	Dataset 1	Zero-Shot	No	Establish baseline
2	Dataset 2	Zero-Shot	No	Quantify Lexical Bias
3	Dataset 2	CoT	No	Evaluate reasoning benefit
4	Dataset 2	Zero-Shot	Yes	Hybrid tool integration
5	Dataset 2	CoT	Yes	Measure Slither Bias

B. Setup

All experiments were performed on a single machine with an M5 chip and 24GB shared RAM/VRAM memory. All local models were run using 4-bit quantization to reduce their memory footprint by decreasing the precision of their weights. No fine-tuning was performed to alter the performance of these local models. The *temperature* parameter was set to 0 and the *timeout* for each LLM query was set to 5 minutes.

C. Model Selection

The primary criterion for model selection was the feasibility of long-term integration into custom static analysis workflows. To keep reasonable financial costs, we fixed a strict pricing ceiling of \$5 per 1 million input/output tokens. This threshold implied the exclusion of flagship models from numerous providers, such as Claude 4.5 Opus (Anthropic), which we categorize as too costly for large-scale auditing tasks.

Following this financial filtering, we selected models specifically to address the objectives of model scaling and architectural comparison. We prioritized models that exist in paired series, allowing for direct comparisons regarding model size and reasoning capabilities. Models that met the pricing criteria but lacked a relevant counterpart for scaling or reasoning analysis were excluded to maintain the thematic consistency of the experimental matrix.

The final list of included models is shown in Table III, and the API providers used to access them are listed in Appendix E, Table VIII.

D. Metrics

We adopt the performance metrics defined in [2]. The presence of positive/negative variations within the test suite is

TABLE III
MODELS AND THEIR DEVELOPERS INCLUDED IN OUR STUDY

Developers	Models
DeepSeek	DeepSeek Chat, DeepSeek Reasoner, DeepSeek R1 14b
Mistral	Magistral Medium, Magistral Small (local and API), Devstral, Devstral Small (local and API)
xAI	Grok 4.1 Fast Reasoning, Grok 4.1 Fast Non-Reasoning
OpenAI	GPT Oss 120b, GPT Oss 20b (local and API)
Alibaba Cloud	Qwen3 30b (thinking), Qwen3 30b (instruct), Qwen3 235b (thinking), Qwen3 235b (instruct)

essential: we compute the False Positive Rate (FPR) and other metrics that quantify the “noise” inherent in LLM outputs.

We only show here *Recall* and *FPR*. Based on these, we compute the Balanced Accuracy (BA) to enable objective comparison across our scenarios and address our research questions.

1) *Recall*: Measures the proportion of actual positive instances that were correctly identified by the model:

$$Recall = TP / (TP + FN),$$

where *TP* is the number of true positives and *FN* is the number of false negatives. Recall is particularly important when the cost of missing a positive instance (a false negative) is high.

2) *False Positive Rate (FPR)*: Represents the proportion of negative instances that are incorrectly classified as positive:

$$FPR = FP / (FP + TN),$$

where *FP* is the number of false positives and *TN* is the number of true negatives. A lower value for *FPR* is better.

3) *Balanced Accuracy*: It is computed as

$$BA = (Recall + (1 - FPR)) / 2,$$

and tells us how well a tool finds positives and avoids false alarms. A higher value for *BA* is better.

E. Results

General experiment-level results (detection metrics, confusion-matrix counts, and reliability per experiment) are summarized in Appendix F, while the per-category results are available in the companion repository referenced in Section VI.

F. Analysis of the Results

Our analysis of the results addresses each research question, with aggregated changes in balanced accuracy reported.

1) *Quantifying the lexical bias between the two datasets*: To answer **RQ1**, we compare Experiment 1 (which uses the original defect examples containing syntactical clues that LLMs might exploit) against all other experiments (which use the preprocessed defects suite with syntactical clues removed).

The raw data shows that Experiment 1 consistently achieves higher BA than Experiments 2–5 (cf. Table IV), indicating the *presence of lexical bias*. This can be seen in Figure 1, where the Zero-Shot blue bars have better scores. This suggests that specific code wording may enable LLMs to detect defects through lexical cues rather than by reasoning about the underlying issue.

2) *Analyzing the impact of using CoT prompting*: To answer **RQ2**, we compare Experiments 2 and 3 (Zero-Shot vs. CoT without Slither) and Experiments 4 and 5 (Zero-Shot vs. CoT with Slither) to assess whether CoT outperforms Zero-Shot. Table V shows that without Slither, CoT performs slightly worse, while with Slither, CoT performs slightly better.

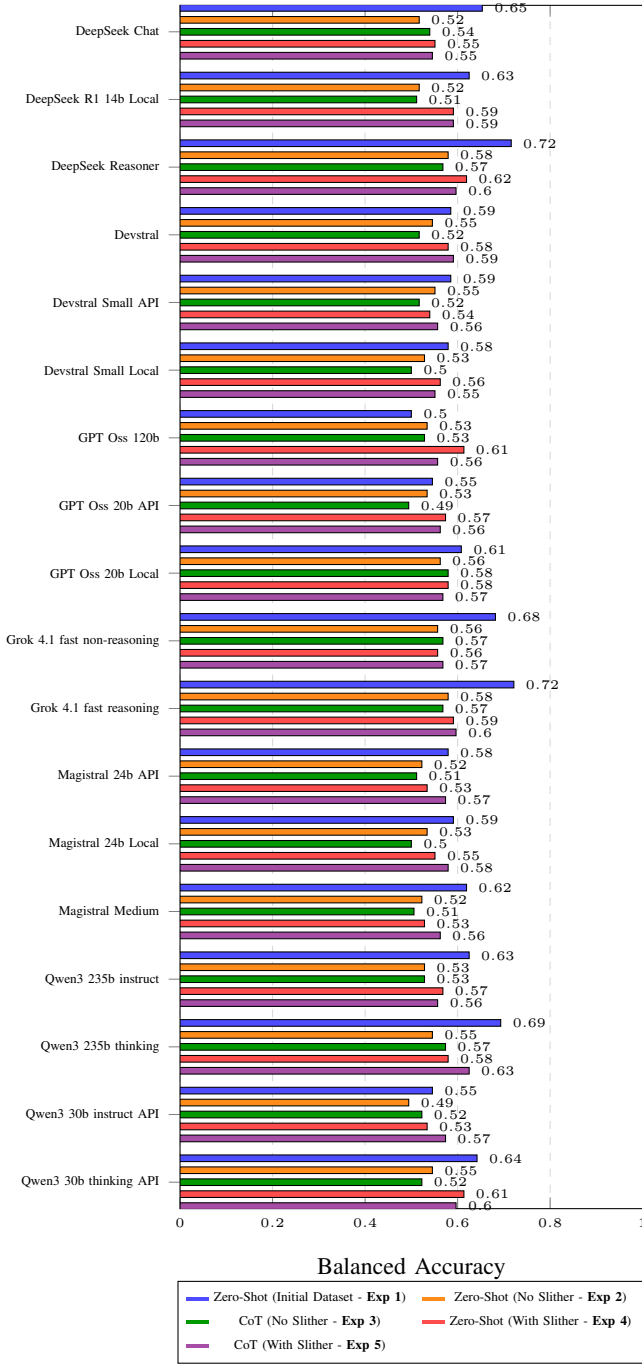


Fig. 1. Balanced accuracy across all experiments.

However, on aggregate, the Δ BA—defined as the numerical difference in BA between two experimental configurations—between CoT and Zero-Shot is within per-model variance (± 0.06).

Notably, the effect of CoT varies across models: for Qwen3 30B Instruct API, Qwen3 235B Thinking, and Magistral 24B API, CoT improves BA by approximately 0.04, while for other models (e.g., GPT Oss 120b) it decreases BA by up to 0.06.

In addition to the above, CoT increased the computational

TABLE IV
COMPARISON OF BA BETWEEN EXPERIMENTS 2–5 AND EXPERIMENT 1. VALUES INDICATE LOWER DETECTION PERFORMANCE IN EXPERIMENTS 2–5 COMPARED TO EXPERIMENT 1, SUGGESTING THAT IN EXPERIMENT 1 THE SUPERIOR ACCURACY IS DUE TO LEXICAL BIAS.

Exp I vs. Exp J	Result	Δ BA	Status
Exp 2 vs. Exp 1	WORSE	-0.077652	
Exp 3 vs. Exp 1	WORSE	-0.085543	
Exp 4 vs. Exp 1	WORSE	-0.046086	
Exp 5 vs. Exp 1	WORSE	-0.041351	

TABLE V
COMPARISON OF BA BETWEEN EXPERIMENTS WITH ZERO-SHOT AND CoT. VALUES DO NOT INDICATE AN IMPROVEMENT WHEN USING CoT.

Exp I vs. Exp J	Result	Δ BA	Status
Exp 3 vs. Exp 2	WORSE	-0.007891	
Exp 5 vs. Exp 4	BETTER	0.004735	

overhead, as models were required to generate an internal chain of reasoning. The time that was necessary for the models to respond increased by up to **10 seconds**. Almost all models queried via API did not suffer shifts in structured output consistency compared to Experiment 2. However, a general increase in invalid outputs was noticed for the local models, the number of such instances increasing by between **10** and **20**, effectively doubling for most models.

3) *Assessing the performance induced by integrating LLMs with Slither:* An answer for **RQ3** requires us to evaluate the performance boost introduced when LLMs are supplemented with external security reports, such as those from Slither. Therefore, we compare Zero-Shot with/without Slither (Exp 2 vs. Exp 4) and CoT with/without Slither (Exp 3 vs. Exp 5).

The results are shown in Table VI. The results indicate that incorporating Slither input significantly improves accuracy.

4) *Assessing the bias induced by grounding LLMs with Slither detections:* To answer **RQ4** we need to compare the matched detections between LLMs and Slither. More precisely, we count the number of LLM detections that match Slither detections in all scenarios. Because this analysis differs from the preceding BA-based evaluations, we report the full results in Figure 2. The number of matching detections is higher when Slither is used.

Without an explicit internal validation step, models in Experiment 4 often adopted Slither’s findings with limited additional scrutiny, leading to substantial increases in matching detections. This reliance on external data was most evident in models like Qwen3 30b Thinking API, which saw a **354%**

TABLE VI
COMPARISON OF BA BETWEEN EXPERIMENTS WITH/WITHOUT SLITHER INPUT. VALUES FOR CoT EXPERIMENTS 4 AND 5 DO INDICATE AN IMPROVEMENT WHEN USING SLITHER INPUT FOR LLMs.

Exp I vs. Exp J	Result	Δ BA	Status
Exp 4 vs. Exp 2	BETTER	0.031566	
Exp 5 vs. Exp 3	BETTER	0.044192	

increase in detections matching Slither’s report compared to its standalone baseline. This suggests that in Zero-Shot mode, the model functions less as an autonomous auditor and more as a parser for the provided deterministic tool output.

In the CoT experiments, models were instructed to follow a structured approach for validating or challenging Slither’s findings through an *adversarial pressure test*. This test required models to determine whether a Slither finding was benign or to identify exploits in functions Slither marked as safe. However, the results indicate that CoT often functioned as a bias amplifier rather than a skeptical filter. Instead of using the reasoning chain to identify false leads, many models used the additional computational steps to generate logical-sounding justifications for the tool’s detections to maintain token-to-token coherence. DeepSeek R1 14b Local demonstrated this clearly, with Slither-aligned detections increasing by **276%** in the CoT hybrid mode. This failure to maintain an independent adversarial stance suggests that the deterministic signal from a known security tool carries a higher weight in the model’s attention mechanism than hypothetical adversarial instructions.

In Zero-Shot, the bias appears consistent with the model’s attention mechanism prioritizing structured, deterministic input (Slither reports) over ambiguous raw code. In CoT, the bias appears to stem from instructional alignment; because the model is tasked with analyzing the report, it may treat agreement with the tool as a signal of task completion. This “Stochastic Mirror” effect raises a significant concern for agentic AI systems. If an autonomous agent cannot effectively question its toolset or identify semantic vulnerabilities that lie outside rigid rule-sets, it cannot provide the adversarial redundancy required for an effective audit process.

5) *The feasibility of running models locally*: Because proprietary contracts cannot be sent to third-party APIs under that on-premises constraint, we compare local 4-bit deployment against full-precision API-based models for security auditing. To answer **RQ5** we evaluate the architectural and operational trade-offs of deploying LLMs locally using 4-bit quantization vs. using full-precision API-based models. Our findings reveal that local deployment is feasible for auditing tasks, but it introduces a class of persistent errors that differentiate them from the transient failures typical of remote services. While API-based models occasionally suffered from issues like server overloads, which can be resolved via automated retries, local models Devstral Small and Magistral 24b exhibited deterministic failures on a recurring subset of contracts that persisted across multiple re-runs. Local failures also manifested as timeouts, driven by the hardware’s inability to complete inference within 5-minute timeout window.

Comparing local and API-based models reveals quantization’s impact on performance:

- 1) Recall drops: API models consistently outperformed their local counterparts; for instance, the API variant of Devstral Small achieved 55.68% recall in Experiment 1, while the local variant dropped to 48.86%.
- 2) False Positive Rates: Local models often exhibited higher noise levels, with the local deployment of Magistral 24b

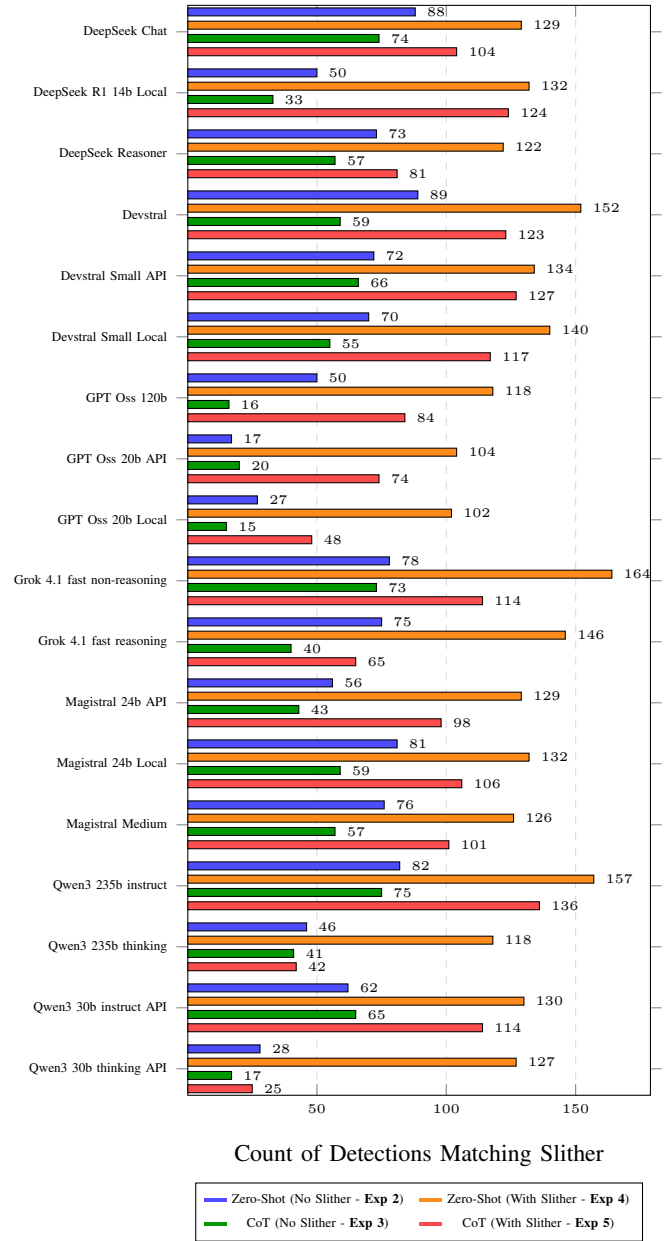


Fig. 2. Slither bias: comparison of matched detections across experiments. Bars show the number of LLM detections that match Slither findings for Zero-Shot and CoT prompting, with and without Slither context.

showing an FPR of 34.09% in Experiment 1 compared to the 25.00% from the API alternative.

- 3) Latency Spikes: The average processing time for local models was significantly higher; for example, Magistral 24b required 44624.74 ms per contract on average for Experiment 3, whereas the API variant required only 2 949.55 ms.

From an architectural standpoint, these discrepancies are largely a consequence of quantization-induced logic loss and hardware constraints. To fit within the 24GB RAM/VRAM limitation of the experimental machine, local models were

compressed to 4-bit precision, which can distort weight distributions necessary for tasks like smart contract auditing. This loss causes the model to be unable to handle tasks involving complex logic, leading to the observed consistency in local errors. Furthermore, the hardware faces extreme pressure when managing the KV cache required for CoT reasoning alongside external Slither reports, leading to a high frequency of invalid or truncated JSON outputs. The mentioned issue regarding KV cache and truncated outputs are the causes of persistent errors. Initially, the errors were presented as connection failures to the local Ollama server. In further investigation, we identified these two issues as the causes of the connection shutdown.

G. Addressing the Automatic Classifier Error Rate

To assess the impact of the corresponding **8%** error rate of our automatic classifier, we identified its likely failure modes.

Our classifier, which depends on semantic similarity, often fails to recognize true positives that are phrased in atypical ways, resulting in their misclassification as false negatives. This primary error mode would artificially lower the reported Recall, Precision, and Accuracy metrics. A secondary, less likely error involves the classifier failing to match a FP from an LLM to any known vulnerability, causing it to be misclassified as a TN. This would lead to an underestimation of the FPR.

With this understanding, we evaluated the robustness of our key findings against a worst-case scenario.

RQ1: Lexical Bias Our finding that **lexical bias** significantly inflates LLM performance remains sound. The transition from the original code (Experiment 1) to the sanitized version (Experiment 2) resulted in a “substantial reduction in Recall for every evaluated model”. Because the classifier’s primary error (TP $\rightarrow$ FN) deflates Recall, the actual performance drop observed in our experiments is likely even more pronounced than reported, confirming the vulnerability of models to identifier obfuscation.

RQ2: Reasoning vs. Pattern Matching Our conclusion that CoT prompting functions primarily as a noise filter is robust. For example, Grok 4.1 Fast Reasoning showed an improvement in FPR at the cost of Recall. Even if the classifier misclassified some atypical findings, the consistent trend of increased caution and reduced discovery range across models validates that CoT shifts the model towards structured validation rather than broad discovery.

RQ3: Hybrid Integration Performance Considering the primary error mode of the classifier, the performance boost recorded when grounding LLMs with Slither reports represents a conservative underestimation of the actual improvement. The distinct upward trend in accuracy remains statistically sound despite this marginal classification noise.

RQ4: Hybrid Integration Bias The “Stochastic Mirror” effect is not threatened by an 8% error margin. We observed massive spikes in detections matching Slither—such as the 354% increase for Qwen3 30b Thinking API—and a 276% increase for DeepSeek R1 14b Local in CoT hybrid mode. Since the secondary failure mode (FP $\rightarrow$ TN) would lead to an underestimation of the FPR, the actual grounding bias and

reliance on external tool outputs are likely even higher than our measurements suggest.

RQ5: Architectural Efficiency The performance and reliability trade-offs for local 4-bit quantized models are driven by physical hardware constraints and logic loss. The persistent errors identified were primarily connection shutdowns or truncated JSON outputs caused by KV cache pressure. As these are infrastructure-level failures rather than semantic misclassifications, the classifier’s error rate does not impact our finding that local deployment introduces a distinct class of non-transient errors.

V. KEY TAKEAWAYS

This section outlines the key overall findings we observed following our experiments. These conclusions address the potential of LLMs and strategies for using them more effectively.

A. Enhanced Grounding for Hybrid Solutions

Directly grounding models with security reporting (e.g., Slither) induces a significant grounding bias, providing little benefit over running analysis tools and LLMs independently. Because models cannot currently function as reliable standalone detection tools, external integrations are mandatory; however, providing high-level findings often leads the model to act as a “stochastic mirror” rather than a critical auditor.

We propose shifting the grounding layer towards deterministic contract representations, such as the Control Flow Graph (CFG) and Static Single Assignment (SSA) forms. While Slither computes this information internally, the current Slither MCP implementation does not expose it through its endpoints. Designed mostly for IDE plugins like GitHub Copilot, it currently only exposes limited data, such as function names and access modifiers, which is insufficient for a deeper analysis.

A further avenue for future work is to reposition the LLM as a property-based test generator for formal verification tools such as Kontrol or Forge.

B. A multi-staged prompting approach

In addition to the required improvements in the grounding mechanism, our results demonstrate that, for agentic security workflows, a single prompting strategy may not suffice. Instead, the most effective systems should adopt a tiered architecture: use Zero-Shot prompting for broad discovery to capture a wide range of potential vulnerabilities, followed by a CoT verification pass to refine the results. This allows the system to maintain a high discovery range (Recall) while using CoT to reduce false positives. By using both approaches and dividing them into different stages, agentic solutions can achieve a more reliable and human-readable audit trail without the performance collapse typically seen in single-pass reasoning.

C. The Quantization Shortcomings on Local Agents

The performance gap between local 4-bit quantized models and full-precision APIs remains a significant limitation

for agentic solutions. While running models locally offers immense financial and privacy advantages, the current trade-off is substantial: local models exhibit persistent, non-transient errors and extreme latency. However, the high cost of accessing top-tier API models remains a practical limitation, making lower-cost models a more viable short-term compromise.

VI. CONCLUSION AND FUTURE WORKS

We presented a large-scale evaluation of 15 LLMs and an automated classification pipeline applied to paired positive/negative Solidity examples. The experiments demonstrate four practical conclusions: LLM detections are sensitive to lexical cues; Chain-of-Thought prompting reduces false positives at the cost of recall and latency; grounding with static-analysis reports increases agreement but can introduce grounding bias; and 4-bit local deployments currently suffer quantization-induced reliability issues.

Taken together, these results indicate that LLMs are not yet reliable as standalone security auditors but can provide value within carefully designed hybrid pipelines. Promising directions include grounding models on deterministic program representations (e.g., CFG/SSA) rather than high-level report text; adopting multi-stage pipelines that separate broad discovery from verification; and improving local-model robustness to mitigate quantization loss. Planned future work includes: • extending the evaluation to real-world contracts (including deployed samples from ecosystems such as Solana), • exposing finer-grained representations from Slither (via an MCP extension), • exploring property-based test generation for formal verifiers, • extending the study to other smart-contract languages such as Rust, and • investigating methods to reduce logic loss in compressed, on-premise models.

Data Availability: We included the contracts for our experiments, additional details regarding prompting, the embeddings used in our automatic classification process, and the location of a web application containing the full results in an anonymous repository: <https://anonymous.4open.science/r/anon-repo-B21A>. Appendix B reproduces the full prompts used in the evaluation.

ACKNOWLEDGMENTS

This work was supported by the *CHIST-ERA-23-SMARTC-01-SCEAL* project, by a grant from the Romanian Ministry of Research, Innovation and Digitization, CNCS/CCCDI - UEFISCDI, project no. 86/2025 ERANET-CHISTERA-IV-SCEAL, within PNCDI IV.

REFERENCES

- [1] Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al.: Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2023)
- [2] Arusoiaie, A., Susan, S.: Towards trusted smart contracts: A comprehensive test suite for vulnerability detection. *Empir. Softw. Eng.* **29**(5), 117 (2024). <https://doi.org/10.1007/S10664-024-10509-W>
- [3] Atzei, N., Bartoletti, M., Cimoli, T.: A survey of attacks on Ethereum smart contracts (SoK). In: Maffei, M., Ryan, M. (eds.) *Principles of Security and Trust*. pp. 164–186. Springer, Berlin, Heidelberg (2017)
- [4] Breidenbach, L., Daian, P., Juels, A., Sirer, E.G.: An in-depth look at the parity multisig bug. <https://hackingdistributed.com/2017/07/22/deep-dive-parity-bug/> (July 2017)
- [5] Buterin, V.: A next-generation smart contract and decentralized application platform. <https://ethereum.org/en/whitepaper/> (dec 2014)
- [6] Chandrasekaran, D., Mago, V.: Evolution of semantic similarity—a survey. *ACM Computing Surveys (Csur)* **54**(2), 1–37 (2021)
- [7] Chen, B., Zhang, Z., Langrené, N., Zhu, S.: Unleashing the potential of prompt engineering in large language models: a comprehensive review (2024). <https://arxiv.org/abs/2310.14735>
- [8] Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., Zheng, Z.: When ChatGPT meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology* (2023)
- [9] Feist, J., Greico, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: *Proceedings of the 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB '19)*, pp. 8–15. IEEE Press, Montreal, Quebec, Canada (2019). <https://doi.org/10.1109/WETSEB.2019.00008>
- [10] Ferreira, J., Durieux, T., Maranhao, R.: Smartbugs wild. <https://github.com/smartbugs/smartbugs-wild> (2020)
- [11] Ferreira, J., Salzer, G.: Smartbugs curated. <https://github.com/smartbugs/smartbugs-curated> (2023)
- [12] Ferreira, J.a.F., Cruz, P., Durieux, T., Abreu, R.: Smartbugs: A framework to analyze Solidity smart contracts. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. p. 1349–1352. ASE '20, ACM, New York, NY, USA (2020). <https://doi.org/10.1145/3324884.3415298>
- [13] Gemini Team: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context (2024). <https://arxiv.org/abs/2403.05530>
- [14] Ghaleb, A., Pattabiraman, K.: Solidifi benchmark. <https://github.com/DependableSystemsLab/Solidifi-benchmark> (2020)
- [15] Grishchenko, I., Maffei, M., Schneidewind, C.: A semantic framework for the security analysis of Ethereum smart contracts. In: Bauer, L., Küsters, R. (eds.) *Principles of Security and Trust*. pp. 243–269. Springer International Publishing, Cham (2018)
- [16] Mense, A., Flatscher, M.: Security vulnerabilities in Ethereum smart contracts. In: *Proceedings of the 20th International Conference on Information Integration and Web-Based Applications and Services*. p. 375–380. iiWAS2018, ACM, New York, NY, USA (2018). <https://doi.org/10.1145/3282373.3282419>
- [17] Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space (2013). <https://arxiv.org/abs/1301.3781>
- [18] NCC-Group: Decentralized application security project. <https://dasp.co/> (2018), accessed: 2023-04-04
- [19] Pennington, J., Socher, R., Manning, C.D.: Glove: Global vectors for word representation. In: *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. pp. 1532–1543 (2014)
- [20] Pinna, A., Ibba, S., Baralla, G., Tonelli, R., Marchesi, M.: A massive analysis of ethereum smart contracts empirical study and code metrics. *IEEE Access* **7**, 78194–78213 (2019). <https://doi.org/10.1109/ACCESS.2019.2921936>
- [21] Rameder, H., di Angelo, M., Salzer, G.: Review of automated vulnerability analysis of smart contracts on Ethereum. *Frontiers in Blockchain* **5** (2022). <https://doi.org/10.3389/fbloc.2022.814977>
- [22] Sharma, N., Sharma, S.: A survey of mythril, a smart contract security analysis tool for evm bytecode. *International Journal of Advanced Research in Computer Science* **13**, 51003–51010 (2022)
- [23] Shiraishi, S., Mohan, V., Marimuthu, H.: Test suites for benchmarks of static analysis tools. In: *2015 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. pp. 12–15 (2015). <https://doi.org/10.1109/ISSREW.2015.7392027>
- [24] Siegel, D.: Understanding the dao attack. <https://www.coindesk.com/learn/2016/06/25/understanding-the-dao-attack/> (July 2016)
- [25] Solidity documentation. <https://docs.soliditylang.org/en/v0.8.16/>, accessed: 2022-09-01
- [26] SunWeb3Sec: Defihacks. <https://github.com/SunWeb3Sec/DeFiHackLabs/> (2023)
- [27] Smart contract weakness classification and test cases. <https://swregistry.io/>, accessed: 2023-04-04

- [28] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023)
- [29] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with ChatGPT. arXiv preprint arXiv:2302.11382 (2023)
- [30] Xiao, Z., Wang, Q., Pearce, H., Chen, S.: Logic meets magic: Llms cracking smart contract vulnerabilities. arXiv preprint arXiv:2501.07058 (2025)
- [31] Xu, Z., Ren, M.: Smart-contract-benchmark-suites: A unified dataset. <https://github.com/renardbebe/Smart-Contract-Benchmark-Suites> (2021)
- [32] Young, A., et al.: Yi: Open foundation models by 01.AI (2025), <https://arxiv.org/abs/2403.04652>
- [33] Zhang, L., Ergen, T., Logeswaran, L., Lee, M., Jurgens, D.: Sprig: Improving large language model performance by system prompt optimization (2024), <https://arxiv.org/abs/2410.14826>
- [34] Zhang, Z., Zhang, B., Xu, W., Lin, Z.: Web3bugs. <https://github.com/ZhangZhuoSJTU/Web3Bugs/tree/main> (2022)
- [35] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *arXiv preprint arXiv:2307.03172*, 2023.
- [36] D. Shrivastava, D. Kocetkov, H. de Vries, D. Bahdanau, and T. Scholak, “RepoFusion: Training code models to understand your repository,” *arXiv preprint arXiv:2306.10998*, 2023.
- [37] P. Shojaei, I. Mirzadeh, K. Alizadeh, M. Horton, S. Bengio, and M. Farajtabar, “The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [38] A. T. Kalai, O. Nachum, S. S. Vempala, and E. Zhang, “Why language models hallucinate,” *arXiv preprint arXiv:2509.04664*, 2025.
- [39] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” in *Proc. 2021 ACM Conf. Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.
- [40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. 30th Conf. Neural Inf. Process. Syst. (NIPS)*, 2017.
- [41] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22128–22142, 2022.
- [42] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A Survey of Quantization Methods for Efficient Neural Network Inference,” *Low-Power Computer Vision*, pp. 291–326, 2021.
- [43] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.
- [44] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [45] R. Pope, S. Douglas, A. Chowdhery, C. Devane, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently Scaling Transformer Inference,” in *Proceedings of the 6th MLSys Conference*, 2023.
- [46] T. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 1877–1901.
- [47] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Proc. 36th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2022, pp. 24824–24837.

APPENDIX A METHODOLOGY FLOWCHART

This diagram provides a compact overview of the end-to-end benchmarking pipeline used in our study, from dataset preparation and prompt configuration to model execution and detection classification. It is intended as a reference map for the experimental workflow discussed in Section III.

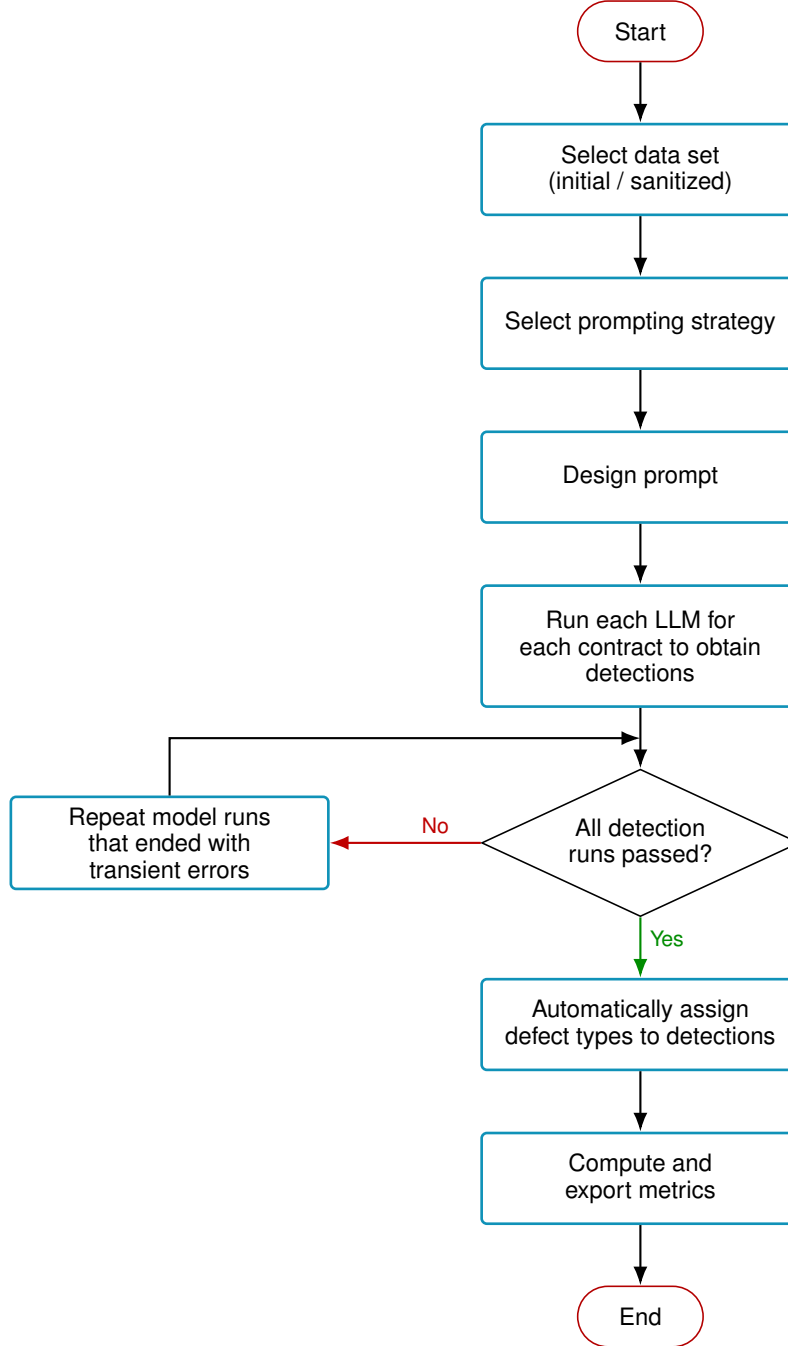


Fig. 3. Methodology flowchart for the automated benchmarking framework.

APPENDIX B PROMPTS USED FOR LLM EVALUATION

The prompts below are reproduced here and include a short description of the intended use of each prompt. The structured output prompt is the system prompt used to ensure schema-conformant output for models without a dedicated request parameter.

A. Zero-Shot Prompt

Baseline prompt without external tool grounding.

```
**Role:** You are a senior Smart Contract Security Auditor specializing in Solidity.  
**Task:** Conduct a deep-dive security analysis of the provided contract(s).  
**Code:**  
[CONTRACT_CODE]
```

B. Zero-Shot + Slither Prompt

Baseline prompt with Slither grounding.

```
**Role:** You are a senior Smart Contract Security Auditor specializing in Solidity.  
**Task:** Conduct a deep-dive security analysis of the provided contract(s).  
**Code:**  
[CONTRACT_CODE]  
**Slither Output:**  
[SLITHER_DETECTIONS]  
Use the Slither output to ground your analysis, but do not limit your findings to these detections; perform a full manual review.
```

C. CoT Prompt

Reasoning-oriented prompt without Slither grounding.

```
**Role:** You are a senior Smart Contract Security Auditor specializing in Solidity and EVM-level exploits.  
**Task:** Conduct a high-precision security analysis of the provided contract(s).  
**Execution Directives:**  
1. Invariant Mapping: Identify the critical security invariants of this contract (e.g., "Total deposits must always equal or exceed the sum of individual balances"). 2. Adversarial State Analysis: Systematically analyze every state-changing function. Determine if a sequence of transactions—potentially involving multiple users or flash-loan-funded interactions—can violate the identified invariants or bypass access controls. 3. Logic-Heavy Search: Prioritize deep logical flaws that traditional static analysis tools often miss, such as flawed economic incentives, incorrect state updates, or integration-specific vulnerabilities (e.g., ERC-20 token standard edge cases like fee-on-transfer or rebasing tokens).  
**Pruning:** Only return findings that represent a genuine, realistically exploitable risk to funds, contract availability, or administrative integrity.  
**Code:**  
[CONTRACT_CODE]
```

D. CoT + Slither Prompt

Reasoning-oriented prompt with Slither grounding.

```
**Role:** You are a senior Smart Contract Security Auditor specializing in Solidity and EVM-level exploits.  
**Task:** Conduct a high-precision security analysis of the provided contract(s).  
**Execution Directives:**  
1. Baseline Triage: I have provided the Slither static analysis output below. Use this to identify immediate "hotspots" in the code. In your internal reasoning, evaluate if these detections are true positives or if the contract's specific business logic renders them non-exploitable. 2. Independent Invariant Mapping: Disregard the Slither output for a moment and independently identify the critical security invariants of this contract (e.g., "The contract must never allow withdrawals that exceed the msg.sender's accounted balance"). 3. The "Blind Spot" Search: Proactively search for high-level logical vulnerabilities that Slither is fundamentally unable to detect, such as flawed economic math, incorrect state updates across multiple transactions, or subtle integration risks with external protocols. 4. Adversarial Pressure Test: Attempt to "disprove" the Slither findings. If Slither flags a re-entrancy vulnerability, can you prove it is actually benign? Conversely, if Slither flags nothing in a critical function like withdraw(), can you find a sequence of transactions that breaks it anyway?  
**Pruning:** Only return findings that represent a genuine, realistically exploitable risk to funds, contract availability, or administrative integrity.  
**Code:**  
[CONTRACT_CODE]  
**Slither Output:**  
[SLITHER_DETECTIONS]
```

E. Structured Output Prompt

System prompt used to enforce JSON-conformant structured output when native schema enforcement is unavailable.

The user will provide input text. Please analyze the content and map the relevant information into the structured JSON format provided below.

```
**JSON Schema:**  
[JSON_SCHEMA]  
**Instructions:** 1. The output must strictly conform to the schema provided. 2. Return ONLY the raw JSON. 3. Do not wrap the response in markdown backticks or provide any explanations. 4. If no relevant data is found for a field, use 'null' or an empty array.
```

APPENDIX C

VULNERABILITY SUBCATEGORIES

The filtered benchmark retains the following defect subcategories, grouped by the high-level taxonomy in Table I.

1 Malicious Env., Transactions or Input

- 1A Reentrancy
- 1B Call To The Unknown
- 1C Exact Balance Dependency
- 1D Improper Data Validation
- 1E Vulnerable DELEGATECALL

2 Blockchain/Environment Dependency

- 2A Timestamp Dependency
- 2B Transaction-Ordering Dependency (TOD)
- 2C Bad Random Number Generation
- 2D Improper Private Variable Secret
- 2E Unpredictable State (Dynamic Libraries)
- 2F Block Hash Dependency
- 2G Block Number Dependency

3 Exception & Error Handling Disorders

- 3A Unchecked Low Level Call or Send Return Values
- ~~3B Unexpected Throw and Revert~~ (excluded due to low classifier accuracy)
- 3C Mishandled Out-Of-Gas Exception
- ~~3D Assert, Require or Revert Violation~~ (excluded due to low classifier accuracy)

4 Denial of Service

- 4A Frozen Ether
- 4B Ether Lost In Transfer
- 4C DoS With Block Gas Limit Reached
- 4D DoS By Exception Inside Loop
- 4E Insufficient Gas Griefing

5 Resource Consumption & Gas Issues

- 5A Gas Costly Loops
- 5B Gas Costly Pattern
- 5C High Gas Consumption Of Variable Datatype Or Declaration
- 5D High Gas Consumption Function Type
- 5E Under-Priced Opcodes

6 Auth. & Access Control Vulnerabilities

- 6A Authorization Via Transaction Origin
- 6B Unauthorized Accessibility Due To Wrong Function Or State Variable Visibility
- 6C Unprotected Self-Destruction
- 6D Unauthorized Ether Withdrawal
- 6E Signature-Based Vulnerabilities

7 Arithmetic Bugs

- 7A Integer Over Or Underflow
- 7B No Division by Zero check
- 7C Integer Division Remainder

8 Bad Coding and Language Specifics

- 8A Type Cast
- 8B Coding Error
- 8C Function Ordering
- 8D Deprecated Source Language Features
- 8F Use Of Assembly
- 8G Incorrect Inheritance Order
- 8H Variable Shadowing
- 8I Single Quotes String
- 8J Dead Code
- 8K Violation Of Checks-Effects-Interaction Pattern
- 8L Inadequate Or Incorrect Logging Or Documentation

9 Environment Configuration Issues

- 9A Missing Short Address Or Zero Address validation
- 9B Outdated Compiler Version
- 9C Floating Or No Pragma
- 9D Token API Violation
- 9F Hardcoded Gas

10 Eliminated/Deprecated Vulnerabilities

- ~~10A Callstack Depth Limit~~ (excluded due to low classifier accuracy)
- 10C Erroneous Constructor Name

APPENDIX D

ALLOWED CLOSE MATCHES FOR CLASSIFICATION

The following close-match pairs were accepted during automatic classification.

TABLE VII
CLOSE-MATCH PAIRS ACCEPTED DURING AUTOMATIC CLASSIFICATION

Exact Match	Alternative
Call To The Unknown	Vulnerable DELEGATECALL
Gas Costly Loops	Gas Costly Pattern
Gas Costly Pattern	Gas Costly Loops
High Gas Consumption Function Type	High Gas Consumption Variable
High Gas Consumption Variable	High Gas Consumption Function Type
Violation Of Checks-Effects-Interaction Pattern	Reentrancy
Vulnerable DELEGATECALL	Call To The Unknown

APPENDIX E

API PROVIDERS USED FOR MODEL ACCESS

The following table lists the API providers used in the study and the models accessed through each provider.

TABLE VIII
API PROVIDERS AND MODELS INCLUDED IN OUR STUDY

API Provider	Models
DeepSeek	DeepSeek Chat, DeepSeek Reasoner, DeepSeek R1 14b
Mistral	Magistral Medium, Magistral Small (API), Devstral, Devstral Small (API)
xAI	Grok 4.1 Fast Reasoning, Grok 4.1 Fast Non-Reasoning
OpenRouter	GPT Oss 120b, GPT Oss 20b (API)
Alibaba Cloud	Qwen3 30b (thinking), Qwen3 30b (instruct), Qwen3 235b (thinking), Qwen3 235b (instruct)
Ollama	DeepSeek R1 14b (local), Devstral Small (local), GPT Oss 20b (local), Magistral 24b (local)

APPENDIX F

EXPERIMENTAL RESULTS

This appendix contains the detailed results for the five experiments. Each experiment is organized into three tables: detection metrics, confusion-matrix counts, and reliability.

The first table for each experiment reports detection metrics.

Model

identifies the evaluated LLM.

Recall

gives the proportion of positive instances correctly identified by the model.

False Positive Rate

gives the proportion of negative instances incorrectly classified as positive.

Balanced Accuracy

gives the average of recall and the true-negative rate.

Precision

gives the proportion of positive predictions that are correct.

The second table reports the confusion-matrix counts used to compute the metrics.

Model

identifies the evaluated LLM.

True Positives

counts positive instances correctly identified as positive.

False Positives

counts negative instances incorrectly identified as positive.

True Negatives

counts negative instances correctly identified as negative.

False Negatives

counts positive instances incorrectly identified as negative.

The third table summarizes reliability and execution behavior.

Model

identifies the evaluated LLM.

Timeouts

counts runs that exceeded the allowed time limit.

Invalid Results

counts instances in which the model did not return output conforming to the expected structure.

Detections

counts the total number of detections produced by the model.

Average runtime in milliseconds

gives the mean response time per contract.

Errors

counts non-transient errors, that is, failures that persisted across retries. In our experiments, transient errors were typically provider-side overloads, whereas non-transient errors were observed mainly in locally deployed models because of hardware limitations on the MacBook M5 system with 24 GB unified memory.

TABLE IX
EXPERIMENT 1 DETECTION METRICS

Model	Recall	False Positive Rate	Balanced Accuracy	Precision
GPT Oss 120b	0.35	0.35	0.5	0.5
GPT Oss 20b API	0.21	0.12	0.54	0.63
GPT Oss 20b Local	0.36	0.14	0.6	0.71
Grok 4.1 fast non-reasoning	0.67	0.3	0.68	0.68
Grok 4.1 fast reasoning	0.68	0.23	0.72	0.74
Devstral	0.57	0.4	0.58	0.58
Magistral Medium	0.52	0.28	0.61	0.64
Magistral 24b API	0.4	0.25	0.57	0.62
Devstral Small API	0.55	0.38	0.58	0.59
Magistral 24b Local	0.52	0.34	0.59	0.6
Devstral Small Local	0.48	0.32	0.57	0.59
Qwen3 235b thinking	0.55	0.17	0.69	0.76
Qwen3 30b thinking API	0.43	0.14	0.64	0.74
Qwen3 235b instruct	0.62	0.37	0.62	0.62
Qwen3 30b instruct API	0.6	0.51	0.54	0.54
DeepSeek Chat	0.69	0.38	0.65	0.64
DeepSeek Reasoner	0.64	0.21	0.71	0.75
DeepSeek R1 14b Local	0.46	0.21	0.62	0.68

TABLE X
EXPERIMENT 1 TP, FP, TN, AND FN COUNTS

Model	True Positives	False Positives	True Negatives	False Negatives
GPT Oss 120b	31	31	57	57
GPT Oss 20b API	19	11	77	69
GPT Oss 20b Local	32	13	75	56
Grok 4.1 fast non-reasoning	59	27	61	29
Grok 4.1 fast reasoning	60	21	67	28
Devstral	51	36	52	37
Magistral Medium	46	25	63	42
Magistral 24b API	36	22	66	52
Devstral Small API	49	34	54	39
Magistral 24b Local	46	30	58	42
Devstral Small Local	43	29	59	45
Qwen3 235b thinking	49	15	73	39
Qwen3 30b thinking API	38	13	75	50
Qwen3 235b instruct	55	33	55	33
Qwen3 30b instruct API	53	45	43	35
DeepSeek Chat	61	34	54	27
DeepSeek Reasoner	57	19	69	31
DeepSeek R1 14b Local	41	19	69	47

TABLE XI
EXPERIMENT 1 RELIABILITY

Model	Timeouts	Invalid Results	Detections	Average runtime (ms)	Errors
GPT Oss 120b	0	47	811	26246.45	0
GPT Oss 20b API	0	24	297	19687.48	0
GPT Oss 20b Local	15	19	335	82885.19	2
Grok 4.1 fast non-reasoning	0	0	803	4527.81	0
Grok 4.1 fast reasoning	0	0	707	21461.32	0
Devstral	0	0	728	7751.04	0
Magistral Medium	0	22	422	10743.78	0
Magistral 24b API	0	13	430	2895.19	0
Devstral Small API	0	5	593	2489.17	0
Magistral 24b Local	0	9	432	39808.92	0
Devstral Small Local	0	11	587	58282	11
Qwen3 235b thinking	0	0	389	72305.21	0
Qwen3 30b thinking API	0	2	211	21818.17	0
Qwen3 235b instruct	0	5	534	6913.48	0
Qwen3 30b instruct API	0	7	420	5116.01	1
DeepSeek Chat	0	0	680	16232.74	0
DeepSeek Reasoner	0	0	478	77573.37	0
DeepSeek R1 14b Local	0	15	233	62654.03	0

TABLE XII
EXPERIMENT 2 DETECTION METRICS

Model	Recall	False Positive Rate	Balanced Accuracy	Precision
GPT Oss 120b	0.26	0.19	0.53	0.57
GPT Oss 20b API	0.15	0.09	0.53	0.63
GPT Oss 20b Local	0.23	0.11	0.56	0.67
Grok 4.1 fast non-reasoning	0.43	0.31	0.55	0.57
Grok 4.1 fast reasoning	0.43	0.27	0.57	0.61
Devstral	0.38	0.29	0.54	0.56
Magistral Medium	0.28	0.23	0.52	0.54
Magistral 24b API	0.23	0.19	0.52	0.55
Devstral Small API	0.32	0.22	0.55	0.59
Magistral 24b Local	0.29	0.22	0.53	0.56
Devstral Small Local	0.3	0.25	0.52	0.55
Qwen3 235b thinking	0.31	0.22	0.54	0.58
Qwen3 30b thinking API	0.19	0.1	0.54	0.65
Qwen3 235b instruct	0.39	0.34	0.52	0.53
Qwen3 30b instruct API	0.28	0.29	0.49	0.49
DeepSeek R1 14b Local	0.22	0.19	0.51	0.54
DeepSeek Chat	0.38	0.35	0.51	0.52
DeepSeek Reasoner	0.37	0.21	0.57	0.63

TABLE XIII
EXPERIMENT 2 TP, FP, TN, AND FN COUNTS

Model	True Positives	False Positives	True Negatives	False Negatives
GPT Oss 120b	23	17	71	65
GPT Oss 20b API	14	8	80	74
GPT Oss 20b Local	21	10	78	67
Grok 4.1 fast non-reasoning	38	28	60	50
Grok 4.1 fast reasoning	38	24	64	50
Devstral	34	26	62	54
Magistral Medium	25	21	67	63
Magistral 24b API	21	17	71	67
Devstral Small API	29	20	68	59
Magistral 24b Local	26	20	68	62
Devstral Small Local	27	22	66	61
Qwen3 235b thinking	28	20	68	60
Qwen3 30b thinking API	17	9	79	71
Qwen3 235b instruct	35	30	58	53
Qwen3 30b instruct API	25	26	62	63
DeepSeek R1 14b Local	20	17	71	68
DeepSeek Chat	34	31	57	54
DeepSeek Reasoner	33	19	69	55

TABLE XIV
EXPERIMENT 2 RELIABILITY

Model	Timeouts	Invalid Results	Detections	Average runtime (ms)	Errors
GPT Oss 120b	0	53	821	30509.31	0
GPT Oss 20b API	0	38	385	38906.93	0
GPT Oss 20b Local	14	26	367	84266.04	1
Grok 4.1 fast non-reasoning	0	0	871	6571.7	0
Grok 4.1 fast reasoning	0	1	822	32698.62	0
Devstral	0	0	908	8258.89	0
Magistral Medium	0	25	573	8050.61	0
Magistral 24b API	0	17	486	4737.48	0
Devstral Small API	0	12	669	3084.46	0
Magistral 24b Local	0	5	566	47013.09	0
Devstral Small Local	0	10	687	65407.06	10
Qwen3 235b thinking	0	0	458	82007.36	0
Qwen3 30b thinking API	0	4	253	25308.17	0
Qwen3 235b instruct	0	8	702	8536.02	0
Qwen3 30b instruct API	0	11	602	7375.38	0
DeepSeek R1 14b Local	0	4	311	67691.12	0
DeepSeek Chat	0	0	831	25226.73	0
DeepSeek Reasoner	0	0	585	86275.03	0

TABLE XV
EXPERIMENT 3 DETECTION METRICS

Model	Recall	False Positive Rate	Balanced Accuracy	Precision
GPT Oss 120b	0.07	0.02	0.52	0.77
GPT Oss 20b API	0.11	0.12	0.49	0.47
GPT Oss 20b Local	0.18	0.02	0.57	0.88
Grok 4.1 fast non-reasoning	0.39	0.26	0.56	0.6
Grok 4.1 fast reasoning	0.25	0.11	0.56	0.68
Devstral	0.27	0.23	0.51	0.53
Magistral Medium	0.25	0.23	0.5	0.51
Magistral 24b API	0.15	0.13	0.51	0.53
Devstral Small API	0.23	0.2	0.51	0.53
Magistral 24b Local	0.18	0.18	0.5	0.5
Devstral Small Local	0.19	0.19	0.5	0.5
Qwen3 235b thinking	0.23	0.09	0.57	0.72
Qwen3 30b thinking API	0.11	0.06	0.52	0.62
Qwen3 235b instruct	0.34	0.28	0.52	0.54
Qwen3 30b instruct API	0.22	0.18	0.52	0.55
DeepSeek Chat	0.37	0.29	0.53	0.55
DeepSeek Reasoner	0.29	0.15	0.56	0.65
DeepSeek R1 14b Local	0.17	0.14	0.51	0.53

TABLE XVI
EXPERIMENT 3 TP, FP, TN, AND FN COUNTS

Model	True Positives	False Positives	True Negatives	False Negatives
GPT Oss 120b	7	2	86	81
GPT Oss 20b API	10	11	77	78
GPT Oss 20b Local	16	2	86	72
Grok 4.1 fast non-reasoning	35	23	65	53
Grok 4.1 fast reasoning	22	10	78	66
Devstral	24	21	67	64
Magistral Medium	22	21	67	66
Magistral 24b API	14	12	76	74
Devstral Small API	21	18	70	67
Magistral 24b Local	16	16	72	72
Devstral Small Local	17	17	71	71
Qwen3 235b thinking	21	8	80	67
Qwen3 30b thinking API	10	6	82	78
Qwen3 235b instruct	30	25	63	58
Qwen3 30b instruct API	20	16	72	68
DeepSeek Chat	33	26	62	55
DeepSeek Reasoner	26	14	74	62
DeepSeek R1 14b Local	15	13	75	73

TABLE XVII
EXPERIMENT 3 RELIABILITY

Model	Timeouts	Invalid Results	Detections	Average runtime (ms)	Errors
GPT Oss 120b	0	136	100	33082.94	0
GPT Oss 20b API	0	19	283	20209.19	0
GPT Oss 20b Local	24	20	130	92362.95	1
Grok 4.1 fast non-reasoning	0	0	616	5859.35	0
Grok 4.1 fast reasoning	0	0	367	30982.92	0
Devstral	0	0	719	19870.96	0
Magistral Medium	0	25	454	9417.3	0
Magistral 24b API	0	32	294	2949.55	0
Devstral Small API	0	6	562	2471.74	0
Magistral 24b Local	0	15	456	44624.74	0
Devstral Small Local	0	15	547	58759.19	15
Qwen3 235b thinking	0	0	267	89767.63	0
Qwen3 30b thinking API	0	0	133	27868.15	0
Qwen3 235b instruct	0	2	570	8527.46	0
Qwen3 30b instruct API	0	9	448	6838.55	0
DeepSeek Chat	0	0	699	20865.5	0
DeepSeek Reasoner	0	0	436	90862.1	0
DeepSeek R1 14b Local	3	5	268	78296.52	0

TABLE XVIII
EXPERIMENT 4 DETECTION METRICS

Model	Recall	False Positive Rate	Balanced Accuracy	Precision
GPT Oss 120b	0.34	0.11	0.61	0.75
GPT Oss 20b API	0.22	0.07	0.57	0.74
GPT Oss 20b Local	0.3	0.14	0.57	0.67
Grok 4.1 fast non-reasoning	0.46	0.35	0.55	0.56
Grok 4.1 fast reasoning	0.45	0.27	0.59	0.62
Devstral	0.47	0.31	0.57	0.6
Magistral Medium	0.36	0.3	0.52	0.54
Magistral 24b API	0.3	0.23	0.53	0.56
Devstral Small API	0.35	0.27	0.53	0.56
Magistral 24b Local	0.32	0.22	0.55	0.59
Devstral Small Local	0.4	0.28	0.56	0.59
Qwen3 235b thinking	0.38	0.22	0.57	0.62
Qwen3 30b thinking API	0.34	0.11	0.61	0.75
Qwen3 235b instruct	0.43	0.29	0.56	0.59
Qwen3 30b instruct API	0.31	0.25	0.53	0.56
DeepSeek Chat	0.51	0.4	0.55	0.55
DeepSeek Reasoner	0.46	0.22	0.61	0.67
DeepSeek R1 14b Local	0.3	0.12	0.59	0.71

TABLE XIX
EXPERIMENT 4 TP, FP, TN, AND FN COUNTS

Model	True Positives	False Positives	True Negatives	False Negatives
GPT Oss 120b	30	10	78	58
GPT Oss 20b API	20	7	81	68
GPT Oss 20b Local	27	13	75	61
Grok 4.1 fast non-reasoning	41	31	57	47
Grok 4.1 fast reasoning	40	24	64	48
Devstral	42	28	60	46
Magistral Medium	32	27	61	56
Magistral 24b API	27	21	67	61
Devstral Small API	31	24	64	57
Magistral 24b Local	29	20	68	59
Devstral Small Local	36	25	63	52
Qwen3 235b thinking	34	20	68	54
Qwen3 30b thinking API	30	10	78	58
Qwen3 235b instruct	38	26	62	50
Qwen3 30b instruct API	28	22	66	60
DeepSeek Chat	45	36	52	43
DeepSeek Reasoner	41	20	68	47
DeepSeek R1 14b Local	27	11	77	61

TABLE XX
EXPERIMENT 4 RELIABILITY

Model	Timeouts	Invalid Results	Detections	Average runtime (ms)	Errors
GPT Oss 120b	0	19	476	22133.51	0
GPT Oss 20b API	0	9	239	17271.72	0
GPT Oss 20b Local	19	31	549	122301.42	3
Grok 4.1 fast non-reasoning	0	0	1212	5824.03	0
Grok 4.1 fast reasoning	0	1	930	24920.38	0
Devstral	0	0	1059	15997.43	0
Magistral Medium	0	14	641	14881.31	0
Magistral 24b API	0	18	680	5328.39	0
Devstral Small API	0	8	812	3155.15	0
Magistral 24b Local	0	12	653	55521.33	0
Devstral Small Local	0	3	856	78918.41	3
Qwen3 235b thinking	0	0	601	91463.6	0
Qwen3 30b thinking API	0	3	397	25569.06	0
Qwen3 235b instruct	0	2	783	9638.89	0
Qwen3 30b instruct API	0	18	821	11242.03	0
DeepSeek Chat	0	0	1015	32165.43	0
DeepSeek Reasoner	1	0	751	94758.32	0
DeepSeek R1 14b Local	0	4	384	70865.82	0

TABLE XXI
EXPERIMENT 5 DETECTION METRICS

Model	Recall	False Positive Rate	Balanced Accuracy	Precision
GPT Oss 120b	0.21	0.1	0.55	0.67
GPT Oss 20b API	0.2	0.07	0.56	0.72
GPT Oss 20b Local	0.21	0.07	0.56	0.73
Grok 4.1 fast non-reasoning	0.35	0.21	0.56	0.62
Grok 4.1 fast reasoning	0.32	0.13	0.59	0.7
Devstral	0.42	0.23	0.59	0.63
Magistral Medium	0.28	0.15	0.56	0.64
Magistral 24b API	0.27	0.12	0.57	0.68
Devstral Small API	0.31	0.2	0.55	0.6
Magistral 24b Local	0.26	0.1	0.57	0.71
Devstral Small Local	0.3	0.2	0.55	0.6
Qwen3 235b thinking	0.32	0.07	0.62	0.8
Qwen3 30b thinking API	0.19	0	0.59	1
Qwen3 235b instruct	0.32	0.21	0.55	0.6
Qwen3 30b instruct API	0.23	0.09	0.57	0.72
DeepSeek Chat	0.36	0.27	0.54	0.57
DeepSeek Reasoner	0.35	0.15	0.59	0.68
DeepSeek R1 14b Local	0.29	0.11	0.59	0.72

TABLE XXII
EXPERIMENT 5 TP, FP, TN, AND FN COUNTS

Model	True Positives	False Positives	True Negatives	False Negatives
GPT Oss 120b	19	9	79	69
GPT Oss 20b API	18	7	81	70
GPT Oss 20b Local	19	7	81	69
Grok 4.1 fast non-reasoning	31	19	69	57
Grok 4.1 fast reasoning	29	12	76	59
Devstral	37	21	67	51
Magistral Medium	25	14	74	63
Magistral 24b API	24	11	77	64
Devstral Small API	28	18	70	60
Magistral 24b Local	23	9	79	65
Devstral Small Local	27	18	70	61
Qwen3 235b thinking	29	7	81	59
Qwen3 30b thinking API	17	0	88	71
Qwen3 235b instruct	29	19	69	59
Qwen3 30b instruct API	21	8	80	67
DeepSeek Chat	32	24	64	56
DeepSeek Reasoner	31	14	74	57
DeepSeek R1 14b Local	26	10	78	62

TABLE XXIII
EXPERIMENT 5 RELIABILITY

Model	Timeouts	Invalid Results	Detections	Average runtime (ms)	Errors
GPT Oss 120b	0	31	368	25170.26	0
GPT Oss 20b API	0	11	181	31532.16	0
GPT Oss 20b Local	17	22	185	91468.28	1
Grok 4.1 fast non-reasoning	0	0	539	3674.8	0
Grok 4.1 fast reasoning	0	0	318	23957.17	0
Devstral	0	1	633	11376.98	0
Magistral Medium	0	16	283	6339.48	0
Magistral 24b API	0	14	274	3189.82	0
Devstral Small API	0	4	416	2031.21	0
Magistral 24b Local	0	12	297	33693.21	0
Devstral Small Local	0	3	436	47081.38	2
Qwen3 235b thinking	0	0	268	101956.21	0
Qwen3 30b thinking API	0	2	93	24519.21	0
Qwen3 235b instruct	0	1	470	7489.71	0
Qwen3 30b instruct API	0	9	274	4488.86	0
DeepSeek Chat	0	0	594	17888.67	0
DeepSeek Reasoner	0	1	363	88136.39	0
DeepSeek R1 14b Local	1	3	312	64570.89	0